

LPH Proposal — Layered Personal Harness for Multi-Domain LLM Agents

作者: Loom Core research thread **日期:** 2026-06-07 **版本:** Proposal v1 (源于 v7.3 implementation plan 的素材重构) **形态:** 提案文档——按 focus dimension 调研现有方法 → 提出 LPH 设计 → 综合对比表 → 诚实优劣分析

Context

本文档把先前的 implementation plan(v7.3)重构为 proposal 形态。读者目标:

- 让审稿人 / 同事 / 自己 快速理解 personal harness 这一问题域的 focus 分布
- 看清 10 类现有方法的 按 focus 分布 的强弱
- 理解 LPH 在哪些 focus 上 填补缺口,在哪些上 正交叠加,在哪些上 诚实承认弱势

详细 phase-by-phase 实施 roadmap 在收尾处给出简要 outline,不是本提案的重点。

1. Problem Statement — 什么是 "Personal Harness"

Personal Harness 定义:wrap 一个 base LLM 的 用户专属 scaffolding,使其行为持久反映:

- 用户在 多个 domain 上的偏好与元规律
- 用户 显式 lock 的事实 / 立场 / 隐私边界
- 用户使用 跨 model / 跨 vendor 时的不变量

它不同于:

- Prompt engineering(单次任务的 prompt 设计)
- Agentic 模型(把 agent 行为训进权重)
- 单 domain skill 库(只覆盖一个领域)

关键属性:必须 跨会话、跨 domain、跨 model。

1.1 当前 personal harness 的两类 ICLR-aligned 失败

Failure F-comp(compositional generalization):用户在 K 个 domain 上累积的元规律(rubric 怎么更新、locked beliefs 怎么解冲突、新 domain 怎么 bootstrap),被 缠在 domain-specific prompt 中。K+1 个 domain 来了,必须从零再学。

Failure F-lock(lock preservation under online updates):用户 lock 的声明(AAPL bullish until Q3、不要给 PM 看仓位、必须引用 reversal_condition),在 vendor system-prompt 升级 / RLHF 漂移 / context 翻转 / 长 context 截断时,被 silently 改动——因为 lock 只是 prompt 字符串,无类型保护。

这两类失败是本提案的 motivation。

2. Eight Focus Dimensions of Personal Harness

把"personal harness 想解决什么"分解成 8 个 独立 focus dimension。后续每个现有方法都按这 8 维评分。

ID	Focus Dimension	含义	理想行为
F1	Within-trajectory self-correction	单次 agent run 内的迭代纠错	错一步 → reflect → 下一步纠正
F2	Cross-session memory accumulation	跨会话的持久记忆	上周说的偏好,今天还记得

ID	Focus Dimension	含义	理想行为
F3	Skill / procedure organization	可复用命名能力的组织	各 domain 有独立可激活的"做法"模板
F4	Context capacity management	超出 context window 的状态管理	1M token 信息可被 paging / retrieve
F5	Cross-domain compositionality	跨 domain 元规律的可组合泛化	在 K 域学会的"偏好如何更新",自动迁到 K+1 域
F6	User-lock preservation	用户 lock 的不可变性	lock 设定后,无论何种 online update,行为不偏
F7	Per-user privacy / state isolation	个人状态本地、不池化、不共享	数据不进训练集,不跨用户共享
F8	Intrinsic model capability	模型本身的 tool-use / 规划 / reasoning 能力	tool-call 准、多步规划稳、context 利用率高

关键观察:F1-F8 *互相正交*——一个方法可以在 F1 满分而在 F5 零分,反之亦然。Personal harness 的 *整体性能* 是这 8 维向量,不是标量。

3. Survey of 10 Existing Harness Approaches

按 *实现路径* 把现有方法分 10 类。每类给:简述 + 强项 focus + 弱项 focus + 关键限制。

M1 — Self-Critique / Reflexion / SELF-REFINE

简述:agent 在单条轨迹内自评 → 重做 → 收敛。Reflexion 在每 episode 末写"反思文本",注入下一 episode 起点。

- **强项:**F1(单轨迹内迭代是其设计目标)
- **弱项:**F2(反思不跨会话持久化)、F3(无 skill 概念)、F5(单轨迹,无跨域)、F6(反思可被后续反思覆盖)、F7(n/a)
- **关键限制:**把 *记忆* 等同于 *上一次反思文本*——反思是 fact,不是 meta-rule

M2 — Voyager / Generative Curriculum + Skill Library

简述:开放世界(Minecraft)中,LLM 生成课程任务 → 完成后把代码作为 skill 入库 → 后续任务可调用。

- **强项:**F3(skill 库设计典范)、部分 F2(skill 持久化)
- **弱项:**F4(skill 库不分 hot/cold)、F5(单域)、F6(新 skill 可覆盖旧 skill,无类型保护)、F7(单用户研究,非多用户)
- **关键限制:**课程是 *单域 domain-specific meta*,无跨域规律层

M3 — Kimi-K2 / Long-Context Agentic Models

简述:1M-2M context + 原生 agent loop(planning + tool use + code exec)。*"记忆"*=放进 context window。

- **强项:**F4(长 context 是核心卖点)、F8(K2 训练强)、部分 F2(in-context recall)
- **弱项:**F3(context 内无 skill 注册)、F5(context 是 blob,无 typed factor)、F6(用户 lock 是 prompt 字符串,后续 token 可覆盖)、F7(session 级,非 typed per-user state)
- **关键限制:**context 越长 ≠ meta 可分解;长度规模不替代结构

M4 — Hermes / Agentic Fine-Tuned Models (Nous Research 3/4)

简述:fine-tune LLM 让它内化 agent 行为——function-calling 准、tool use 稳。

- **强项:**F8(强 tool use 与 role-play)
- **弱项:**F2/F3/F5/F6/F7 全 ✗——纯模型,无 harness 层,无 per-user 概念
- **关键限制:**权重一体,所有用户共享同一行为

M5 — MemGPT / Hierarchical Memory

简述:模拟操作系统 paging——hot(in-context)/ cold(disk),LLM 自己 swap。

- **强项:**F4(分页是核心贡献)
- **弱项:**F3(page 是 text,非 skill)、F5(无 typed factor)、F6(page swap 可丢 lock)
- **关键限制:**解决 容量,不解决 结构

M6 — RAG + Vector Store

简述:把记忆作为 embeddings 存储,查询时 retrieve top-k 注入 context。

- **强项:**F2(可扩展记忆)、F4(查询时按需注入)
- **弱项:**F3(retrieval 不是 skill 激活)、F5(embedding 无 typed factor)、F6(无 typed lock)、F1(retrieval 不参与轨迹内迭代)
- **关键限制:**retrieval 是 相似度 而非 规则匹配——meta-rule 无法 retrieve

M7 — Superpowers / Claude Skills / Custom GPTs / Cursor Rules

简述:平铺 skill 注册表 + 触发时注入对应 markdown 到 system prompt。

- **强项:**F3(skill 库组织典范)、F2(skill 持久存在文件系统)、F7(本地文件)
- **弱项:**F5(每个 skill 独立,跨 skill 无 typed meta;只能靠人写 CLAUDE.md 总结)、F6(lock 都是 markdown 文本,可被后续 prompt 覆盖)、F4(无分页,skill 全文注入)、F1(无轨迹内反思机制)
- **关键限制:**meta-rule 与 domain-procedure 住在同一个 markdown blob 中,无显式 factor

M8 — Agentic RL (RLVR / ReST / Agentic DPO)

简述:用合成 trajectory 训练 reward / DPO,让模型在 agentic benchmark 上表现更好。代表:DeepSeek-R1、Tulu-3 agentic、ReSTEM。

- **强项:**F8(intrinsic capability 提升)、部分 F5(权重隐式 meta)
- **弱项:**F2(无 per-user 记忆)、F6(权重 RLHF 漂移可静默改 user-locked 行为)、F7(训练数据池化,无 per-user 隔离)、F3(无 skill 注册表)
- **关键限制:**改变 *capability ceiling*,不改 *per-user invariance*

M9 — Active Lessons-Learned Notepad (CLAUDE.md / ChatGPT memory)

简述:产品界最强 baseline——用户告诉 agent"下次不要这样",agent 把这条写进 notepad,下次注入。

- **强项:**F2(简易记忆)、F3(procedure-like notes)、F7(本地存储)、即时部署
- **弱项:**F5(notes 是 *fact*,不是 *meta-rule*;K 域需要 K 份 note,无组合)、F6(string lock 可被后续 prompt 覆盖)、F1(无轨迹内反思)、F4(notepad 越长越占 context)
- **关键限制:**失败 不是质量不够,是 结构性的——fact / meta-rule / lock 全塞进同一个字符串 blob

M10 — Constitutional AI / RLHF Constitution

简述:文本 constitution(行为准则)+ RLHF reward model 训练,让模型内化 alignment 规则。

- **强项:**F6(alignment-time 约束;部分对抗权重漂移)、F8(改善能力)
- **弱项:**F5(单一 constitution,非跨域)、F2(无 per-user 记忆)、F7(population-level,非 per-user)
- **关键限制:**constitution 是 全局 共享的,无 *per-user layer*

4. Cross-Method Gap Map(综合对比表 — 提案核心)

图例:  = 该 focus 是设计目标且实现充分;  = 部分覆盖或依赖人工配置;  = 不在设计目标内或结构性缺失。

方法	F1 within-trajectory	F2 cross-session memory	F3 skill organization	F4 context capacity	F5 cross-domain composition	F6 lock preservation	F7 per-user privacy	F8 intrinsic capability
M1 Reflexion / SELF-REFINE	✔	✗	✗	✗	✗	✗	n/a	⚠
M2 Voyager	⚠	⚠	✔	✗	✗	✗	n/a	⚠
M3 Kimi-K2	⚠	⚠	✗	✔	✗	✗	⚠	✔
M4 Hermes	✗	✗	✗	✗	✗	✗	✗	✔
M5 MemGPT	✗	⚠	✗	✔	✗	✗	⚠	⚠
M6 RAG	✗	✔	✗	✔	✗	✗	⚠	⚠
M7 Superpowers / Skills	✗	⚠	✔	✗	✗	✗	✔	⚠
M8 Agentic RL	⚠	✗	✗	✗	⚠	✗	✗	✔
M9 Notepad	✗	✔	⚠	✗	✗	✗	✔	⚠
M10 Constitutional AI	✗	✗	✗	✗	✗	⚠	✗	✔
LPH (本提案)	✗	✔	✔	⚠	✔	✔	✔	✗

关键观察

Gap 1:F5 列 没有任何已有方法 ✔——所有 10 个方法在跨域 compositional generalization 上都失败,要么因为单域设计 (Voyager),要么因为 meta 缠在 blob 中(Kimi/Skills/Notepad),要么因为不是 personal 范畴(Hermes/Agentic-RL/Constitutional)。

Gap 2:F6 列 没有任何已有方法 ✔——所有 lock 都是字符串 / 权重,无类型保护。最接近的 Constitutional AI 也只能 ⚠,因为它的约束是 全局的,不是 per-user 的。

Gap 3:F5 ∧ F6 ∧ F7 同时 ✔ 的方法不存在——LPH 是第一个填补这个洞的。

LPH 的诚实弱势:

- F1:LPH 没有 within-trajectory 反思机制(Reflexion 强)
- F4:LPH 的 Pack 不内置 paging(MemGPT 强)
- F8:LPH 是 协议层,不改 LLM 本身(Kimi/Hermes/Agentic-RL 强)

→ LPH 在这三列必须诚实承认弱势,并 显式 compose 其它方法填补(详见 §7)。

5. LPH Design

5.1 核心分解(typed factoring)

Brain side — 跨 domain、跨 model swap 持久

- `Brain.meta`:typed family of meta-policies = {`rubric_update`, `lock_resolve`, `new_domain_bootstrap`, `schema_evolution`, `routing_decay`, `presentation_pref`, `cross_domain_rule`}

- `Brain.state`:typed CIR graph,节点类型 = {`Rubric-weight`, `Locked-belief`, `Routing-rule`, `Cross-domain-rule`, `Preference`}

Hand side — domain-specific,可 product-shipped 或 user-evolved

- `Hand.manifest.seed`:product-shipped = {`procedure`, `tools`, `output_schema`, `resource_catalog`}
- `Hand.manifest.evolved`:user-accumulated refinements per domain

5.2 三个纯函数

```
Pack    : (Brain.state, Hand.manifest, Task) → TaskEnvelope
Lift    : (Interaction, Workspace) → Signals
Evolve  : (Brain.meta, Signals, Brain.state, Hand.evolved) → (Δstate, Δevolved)
```

三者均 纯函数,无外部副作用——这是后续 type-level invariance 的形式化基础。

5.3 工作循环(10-step)

```
Task → Brain
  → Pack(state, hand.manifest, task) → TaskEnvelope
  → Hand 执行(不见 Brain.state 全貌)
  → artifact → workspace
  → 用户交互 → Lift → Signals
  → Evolve(meta, signals, state, evolved) → (Δstate, Δevolved)
  → 应用 Δ → 下次 Pack 输出不同 envelope
```

5.4 为什么设计 结构性 填补 F5+F6

F5(cross-domain compositionality):

- `Brain.meta` 作为 typed family,VC-dim 有界
- K 个 domain 交互 → `Brain.meta` 收敛;第 $K+1$ 域 cold-start 时元规律已具备
- 可证: $L(\text{meta}_K, d_{\{K+1\}}) - L(\text{meta_oracle}, d_{\{K+1\}}) \leq C \cdot \log K / K$

F6(lock preservation):

- `Brain.state.Locked-belief` 是 *typed graph* 节点,不是字符串
- `Evolve` type signature 禁止 写 `Locked-belief` 节点
- 唯一可改 lock 的路径:lock_resolve meta-policy + explicit 用户触发
- 可证:任意 online signal 序列下,所有原有 lock 的 value 不变

这两条是 type-level 不变性——不可被字符串 blob 方法功能替代。

6. LPH Pros & Cons(诚实评估)

6.1 优势(F5、F6、F7)

优势	来源	是否可证
跨域 compositional generalization	typed Brain.meta + bounded VC-dim	✅ $O(\log K / K)$ cold-start bound
User-lock preservation	typed Brain.state + Evolve 写域限制	✅ 0% violation(adversarial 1000 signals)
Per-user privacy	Brain.state 本地 typed graph	by design

优势	来源	是否可证
Layered substitutability	Brain × Hand 双层	可换 Hand backend 不丢 Brain.state
Auditability	typed graph Δ 可视化	by design

6.2 劣势(F1、F4、F8)

劣势	谁更强	缓解策略
无 within-trajectory 反思	Reflexion / SELF-REFINE	Hand 内嵌 Reflexion-style loop
无 context paging	MemGPT	Pack inject 阶段叠加 MemGPT
不改 LLM 能力上限	Kimi-K2 / Hermes / Agentic RL	任意 model 作 Hand backend
部署需基础设施	Notepad / Skills 写 markdown 即用	Phase A 提供最小 typed storage
Brain.meta 收敛慢	Notepad 第一次即生效	default seed library bootstrap
初期 Hand.evolved 为空	Skills 自带高质量 procedure	seed = product-shipped skills

6.3 LPH 不适用的场景

场景	推荐方法
单 domain、一次性任务	Notepad / 直接 prompt
不在意 lock,只要"差不多记得"	Notepad / ChatGPT memory
需要 model 能力跃迁	Agentic RL / 更强底座
单次复杂多步任务	Reflexion + 强底座
容量受限(>200K context)	MemGPT 或叠加

7. Orthogonality Map — LPH 如何与其它方法叠加

LPH 并不替代 M1-M10——它是 *typed factoring layer*,在它们之上提供 invariance + cross-domain composition。

LPH ×	叠加方式	收益
× M1 Reflexion	Reflexion 嵌入 Hand 内的 LLM loop	F1 + F5 兼得
× M2 Voyager	Voyager 课程作为 <code>new_domain_bootstrap</code> meta-policy	自动课程能力被 LPH 跨域复用
× M3 Kimi-K2	Kimi 作 Hand backend(http×openai)	F4+F8 由 Kimi 承担,LPH 提供 F5+F6
× M4 Hermes	Hermes 作 tool-use 密集 domain 的 Hand	F8 高 Hand 用于高精度场景
× M5 MemGPT	MemGPT 实现 Pack 的 context budget	F4 由 MemGPT 解决
× M6 RAG	Vector store 作 routing-rule backing	F2 scalability 由 RAG 解决
× M7 Skills/Superpowers	每个 Skill → <code>Hand.manifest.seed</code>	F3 由 Skills 解决,LPH 加跨 Skill 元规律
× M8 Agentic RL	RL-trained model 作 Hand backend	F8 由 RL 承担,LPH 提供独立于权重漂移的 invariance
× M9 Notepad	Notepad 作 Brain.state.Preference 初始化 seed	CLAUDE.md 内容平滑迁移到 typed state

LPH ×	叠加方式	收益
× M10 Constitutional AI	Constitution → 默认 Brain.meta 的固定子集	Global alignment 与 per-user invariance 共存

核心定位:LPH = *invariance* + *factoring layer*。Capability / capacity / reflexion 层正交叠加。

8. Relation to Existing Loom Codebase

复用已有:

- loom_core/agent_adapters/adapter.py:7-provider 抽象 → 直接作 Hand backend layer
- loom_core/runtime/app.py(LoomCoreRuntime):feedback_store / hand_config_store → Lift 信号采集与 Evolve 状态写回
- loom/hands/base.py(BaseHand):继续作 Hand 内部执行循环;envelope 改为 TaskEnvelope 形态
- loom/brain.py FastAPI /run:Pack/Lift/Evolve 接入点

新增独立目录:loom_core/lph/(现不存在)

- brain_state.py / brain_meta.py / hand_manifest.py — typed schema
- pack.py / lift.py / evolve.py — 三纯函数
- meta_policies/ — 默认 meta-policy 库

双协议并行:旧 task+context envelope 保留(compatibility);新 task_envelope 走 LPH;feature flag 切换。不动 anchor-client.js、styles.css、loom-fin。

9. 简要实施 Outline

Phase	内容	周期
A	typed storage skeleton + JSON export	2-3 周
B	Pack/Lift/Evolve + default meta-policy library	3-4 周
C	F6 实验框架 + 1000 对抗信号	3-4 周
D	F5 实验:6-8 domain hold-out + $O(\log K / K)$ 拟合	5-6 周
E	F5+F6 + auditability user study(N=50)	4-5 周
F	理论证明 paper(并行)	6-8 周

目标投稿:ICLR 2027 main / NeurIPS 2026 main / COLM 2026。

10. References

- Shinn et al. "Reflexion"(2023); Madaan et al. "Self-Refine"(2023)
- Wang et al. "Voyager"(2023); Park et al. "Generative Agents"(2023)
- Moonshot AI "Kimi-K2 Technical Report"; Nous Research "Hermes 3/4"
- Packer et al. "MemGPT"(2023); Lewis et al. "RAG"(2020)
- DeepSeek-AI "DeepSeek-R1"(2025); AI2 "Tulu 3"(2024)
- Bai et al. "Constitutional AI"(2022, Anthropic)
- Finn et al. "MAML"(2017); Nichol et al. "Reptile"(2018)

一句话总结

Current LLM personal-harness designs cover at most 4-5 of the 8 focus dimensions. No existing method covers **F5 (cross-domain compositionality)** and **F6 (lock preservation)** simultaneously — both have ICLR-relevant ML/alignment significance. LPH proposes a *typed factoring* of harness state into **Brain (meta + state) × Hand (seed + evolved)**, connected by three pure functions (**Pack / Lift / Evolve**), filling the F5+F6 hole via *type-level* invariants: $O(\log K / K)$ cold-start bound (Theorem 2) and 0% lock-violation under arbitrary online signals (Theorem 1). LPH composes orthogonally with Reflexion (F1) / MemGPT (F4) / Agentic-RL (F8) — it does not compete with them. Honest trade-off: LPH requires $K \geq 3$ domains and deployment infrastructure to show its value.